

Pemrograman Berorientasi Objek

Dany Candra Febrianto, S.Kom., M.Eng.

Minggu 09-10 : Web Dev dengan Spring Boot Part 1
REST API dengan Spring Boot Part 2

*Apa yang kita lakukan dalam hidup akan bergema secara
abadi*

- Maximus Decimus Meridius, (Gladiator, 2000) -

**Jika kamu tidak tahan lelahnya belajar, maka
kamu harus tahan menanggung perihnya
kebodohan**

- Imam Asy-Syafi'i-

Catatan Penting sebelum belajar

- Pastikan memahami Java Dasar
- Pastikan memahami 4 pilar OOP
- Pastikan memahami penggunaan IDE

Content

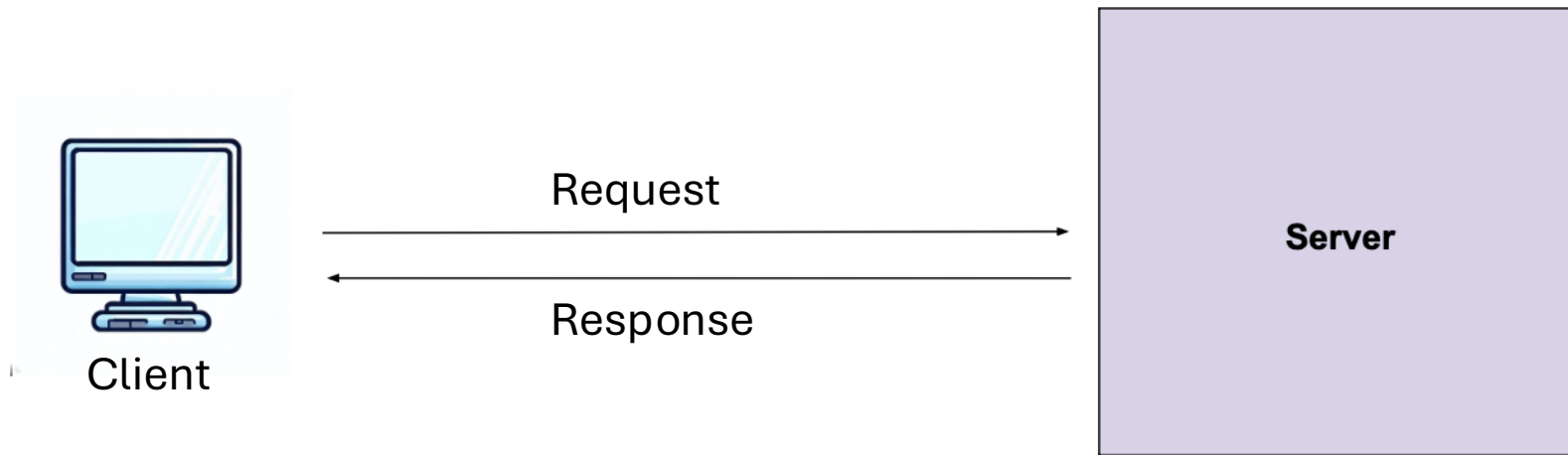
- Konsep Web Application
- Arsitektur MVC
- Implementasi MVC dengan Spring Boot

Konsep Web Application

Apa itu Web Application

Internet	Web
Internet adalah jaringan komputer global yang terhubung	World Wide Web adalah cara mengakses informasi melalui media Internet.

Alur kerja web

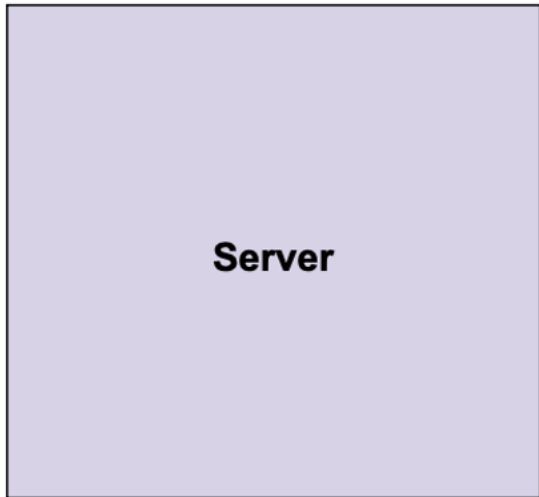


Client



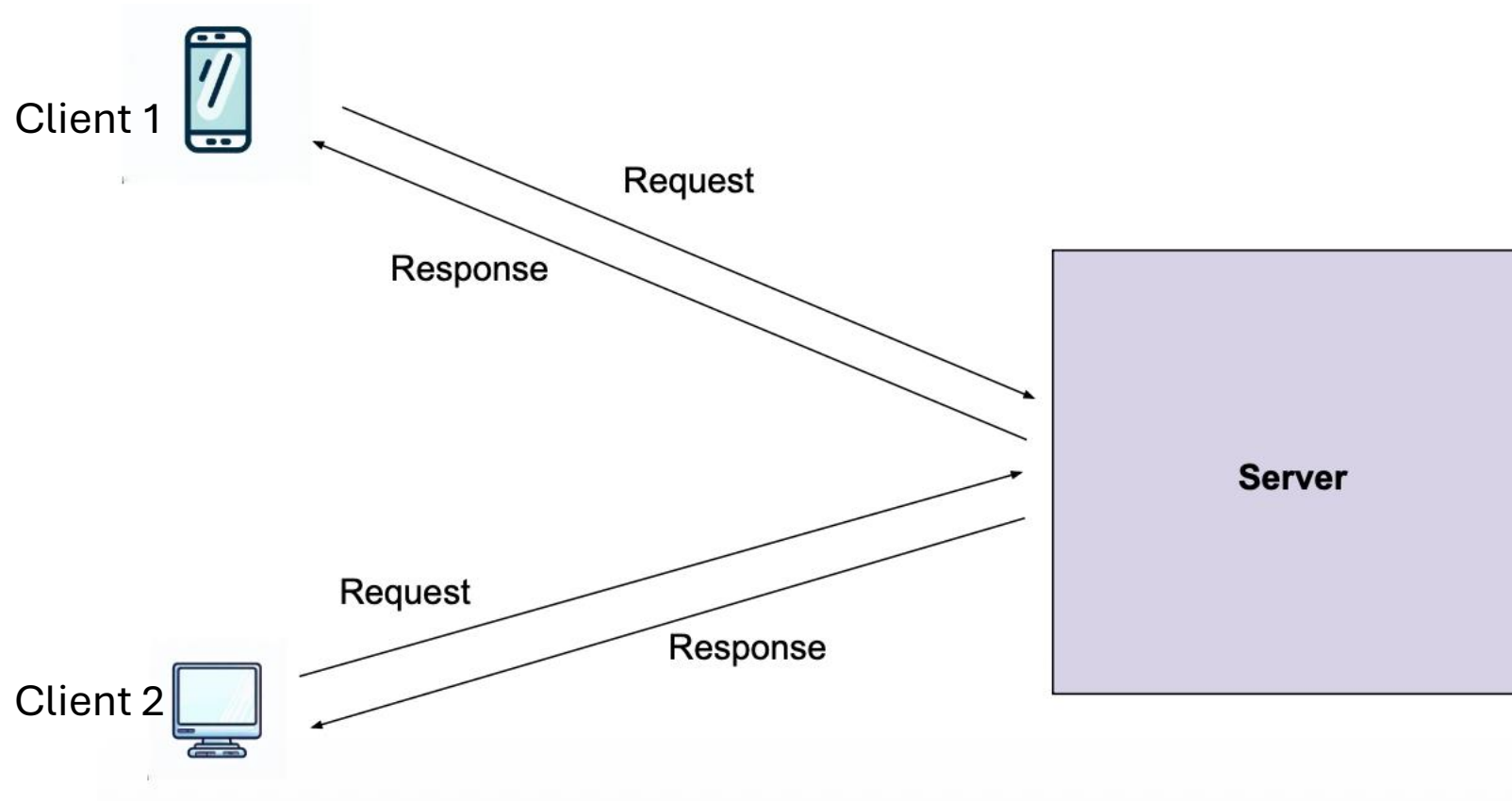
- Client biasanya berupa device / aplikasi yang mengirim *request* berupa *services* atau *resources* kepada server
- Karakter Client
 - Berupa User Interface (Web app / Mobile app)
 - Mengirim Request (Service/resources)
 - Menerima Response (Data : Service / resources)

Server



- Server adalah komputer atau aplikasi yang menyediakan *services* untuk pengguna lain (client). Server menerima permintaan dari client, memprosesnya, kemudian mengirimkan kembali hasilnya. Pada aplikasi web, server menyimpan website dan menampilkan halaman ketika pengguna mengaksesnya melalui browser.
- Karakter Server
 - *Always On*
 - Menangani banyak request
 - Mengirim Response (Data : Service / resources)

Interaksi Client Server



HTTP (Hypertext Transfer Protocol)

- HTTP singkatan dari Hypertext Transfer Protocol
- HTTP merupakan protokol untuk melakukan transmisi hypermedia document, seperti HTML, JavaScript, CSS, Image, Audio, Video dan lain-lain

HTTP Request

HTTP adalah protokol yang digunakan untuk komunikasi antara **client** dan **server** pada aplikasi web.



```
GET /students HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0
Accept: application/json
```

- **GET** → method request
- **/students** → endpoint yang diminta
- **Host** → alamat server
- **Accept** → format data yang diinginkan

HTTP Response

HTTP Response adalah **balasan dari server setelah memproses request**.



```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 54

{"id": 1, "name": "Budi"}
```

- **200 OK** → status request berhasil
- **Content-Type** → jenis data yang dikirim
- **Body** → data hasil response

Request & Response

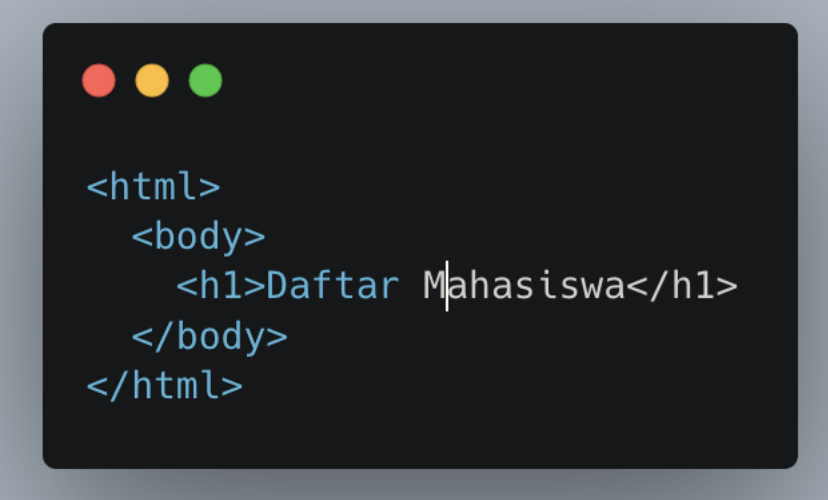
Type Request	Response Code
GET Request	1xx (Informational)
POST Request	2xx (Successful)
PUT Request	3xx (Redirection)
DELETE Request	4xx (Client Error)
	5xx (Server Error)

Web Page & REST API

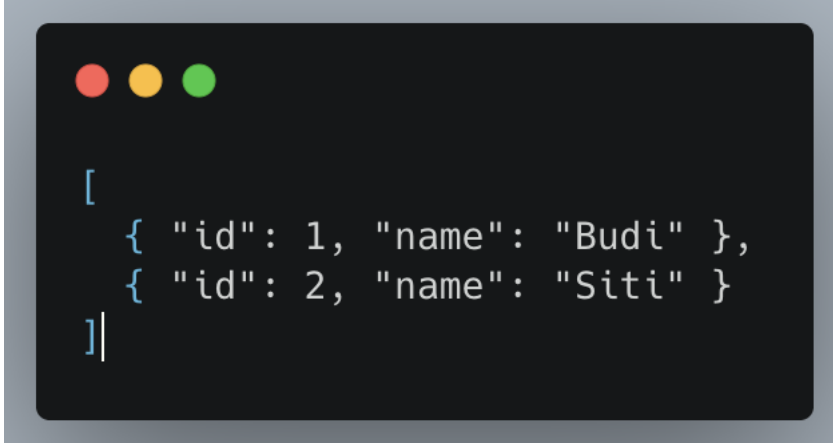
Aspek	Web Page	REST API
Tujuan	Menampilkan halaman web kepada pengguna	Menyediakan data untuk aplikasi lain
Format Response	HTML	JSON / XML
Client	Browser	Browser, mobile app, frontend framework
Tampilan UI	Dibuat di server	Dibuat di client
Contoh Penggunaan	Website	Mobile app, SPA, frontend JS

Format Response

Web Page	REST API
Berupa HTML	Berupa JSON



```
<html>
  <body>
    <h1>Daftar Mahasiswa</h1>
  </body>
</html>
```



```
[
  { "id": 1, "name": "Budi" },
  { "id": 2, "name": "Siti" }
]
```

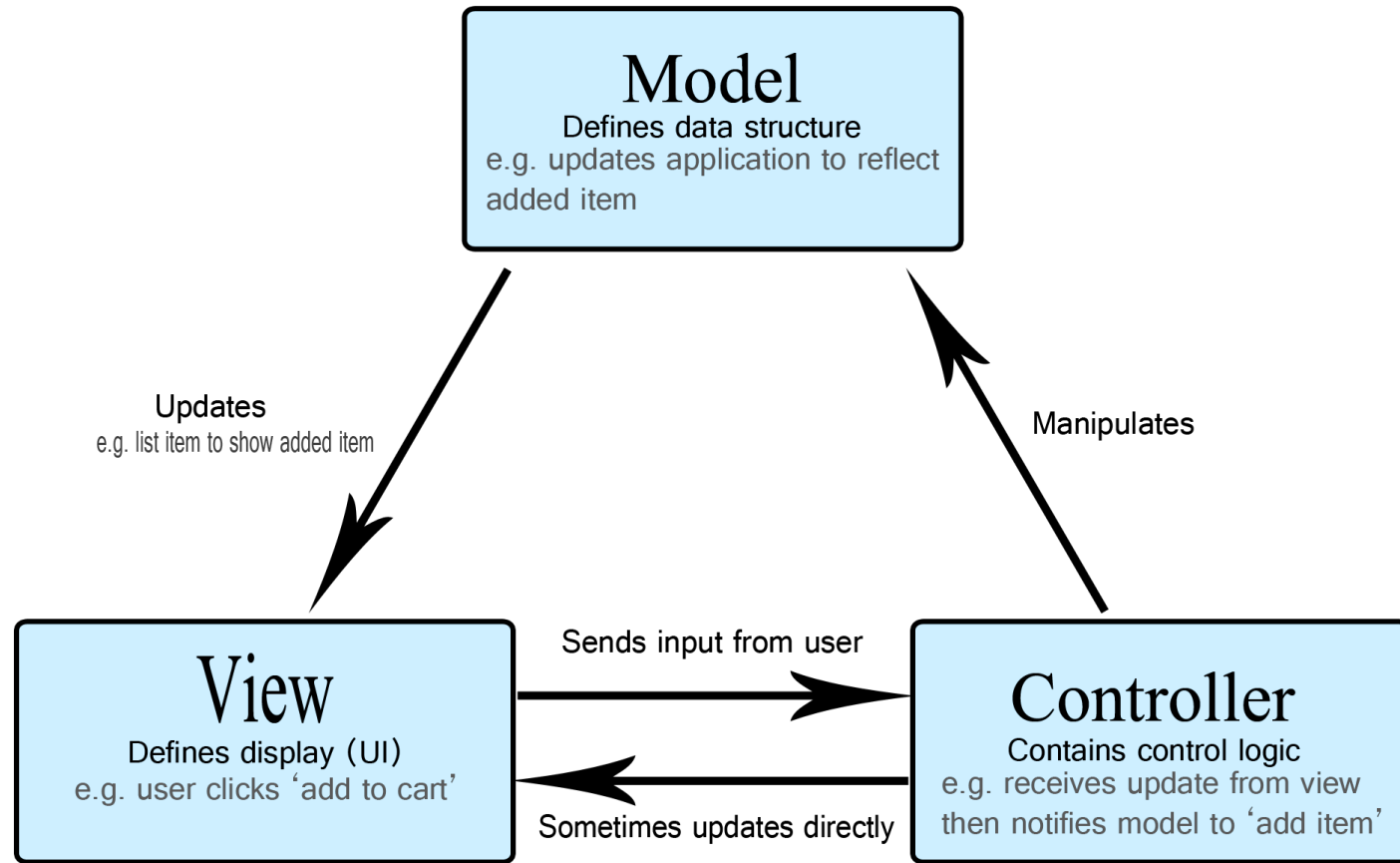
Cara Mengakses

Web Page	REST API
<code>http://localhost:8080/students</code>	<code>http://localhost:8080/api/students</code>
Client hanya menampilkan halaman .	Client harus mengolah data dan membuat UI sendiri .

MVC (Model View Controller)

MVC

- MVC singkatan dari Model View Controller, yaitu salah satu software design pattern yang banyak digunakan ketika pengembangan aplikasi berbasis user interface
- MVC pertama kali dikenalkan oleh Trygve Reenskaug pada tahun 1970 ketika berkunjung ke Xerox Palo Alto Research
- Awalnya MVC banyak digunakan di aplikasi berbasis Desktop, namun sekarang MVC banyak diadopsi di Web



Sumber <https://developer.mozilla.org/en-US/docs/Glossary/MVC>

Model

- Model bertanggung jawab untuk ***mengelola data*** dan logika bisnis. (Biasanya representasi dari Table di Database)
- Contoh tugas model:
 - menyimpan data
 - mengambil data dari database
 - memproses data

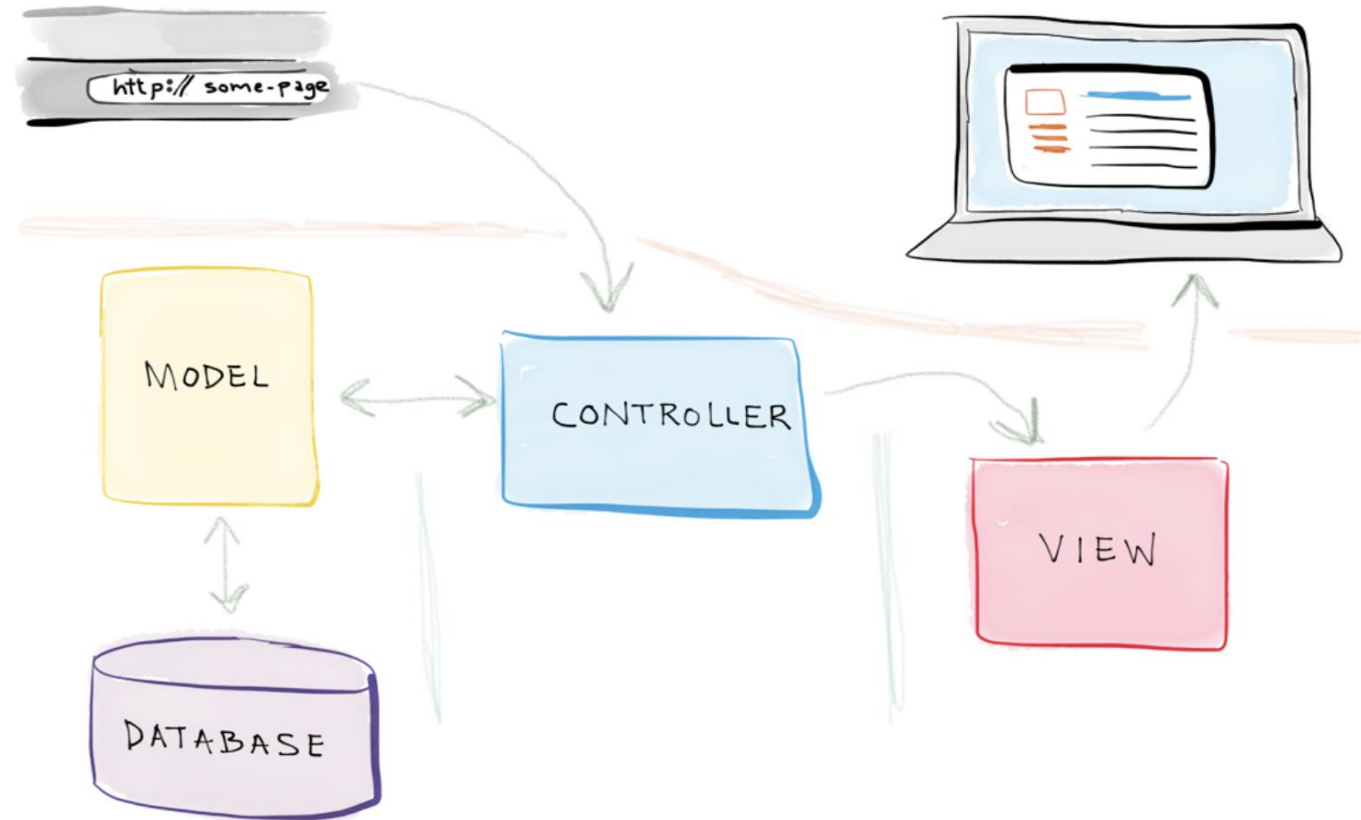
View

- View bertanggung jawab untuk ***menampilkan data kepada pengguna.***
- Biasanya berupa:
 - halaman HTML
 - Java Swing (Desktop)

Controller

- Controller bertugas ***menerima request dari client dan menentukan respon yang diberikan.***
- Controller akan:
 - menerima request
 - memproses request
 - mengambil data dari model
 - mengirim data ke view

MVC di Web



The flow of a request in an MVC web application

Pada Kenyataannya

- Walaupun sekilas MVC sangat sederhana, pada kenyataannya ketika kita membuat aplikasi yang kompleks, kita biasanya tidak lagi bisa memanfaatkan MVC
- Kadang kita butuh mengimplementasikan design pattern lain, seperti misal nya Service Pattern, Repository Pattern, dan lain-lain
- Oleh karena itu, jangan terlalu terpaku pada satu pattern, jika kita bisa mengkombinasikan beberapa pattern agar kode aplikasi kita lebih rapi dan baik, maka disarankan untuk melakukan kombinasi

(source : Programmer Zaman Now)

Implementasi Web MVC dengan Spring Boot

Apa itu Spring Boot

Spring Boot adalah framework Java berbasis [Spring Framework](#) yang menyederhanakan pengembangan aplikasi *stand-alone* dan tingkat produksi dengan konfigurasi otomatis (*opinionated*).

Ini mengurangi kode *boilerplate* dan pengaturan manual (seperti server tertanam seperti [Tomcat](#)). Sangat populer untuk membangun microservices, REST API, dan aplikasi web dengan cepat.

(Source : spring.io)

Spring boot = **Spring Framework**
+
Prebuilt Configuration
+
Embedded Servers

Komponen Spring Boot

- Spring Boot Starters
- Auto Configuration
- Spring Boot Actuator
- Embedded Server
- Spring Boot DevTools

Keuntungan Menggunakan

- Stand alone dan Quick Start
- Starter code
- Minim configuration
- Mengurangi cost dan development time

Cara Membuat Project Spring Boot

- <https://start.spring.io/>
- Tambah Dependency
 - Spring Web
 - Thymeleaf



Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (M2)

☐ 4.0.4 (SNAPSHOT)

☒ 4.0.3

☐ 3.5.12 (SNAPSHOT)

☐ 3.5.11

Project Metadata

Group

com.tup

Artifact

demo

Name

Demo MVC

Description

Demo project for Spring Boot

Package name

com.tup.demo

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Dependencies

ADD DEPENDENCIES... ⌘ + B

Spring Web

WEB

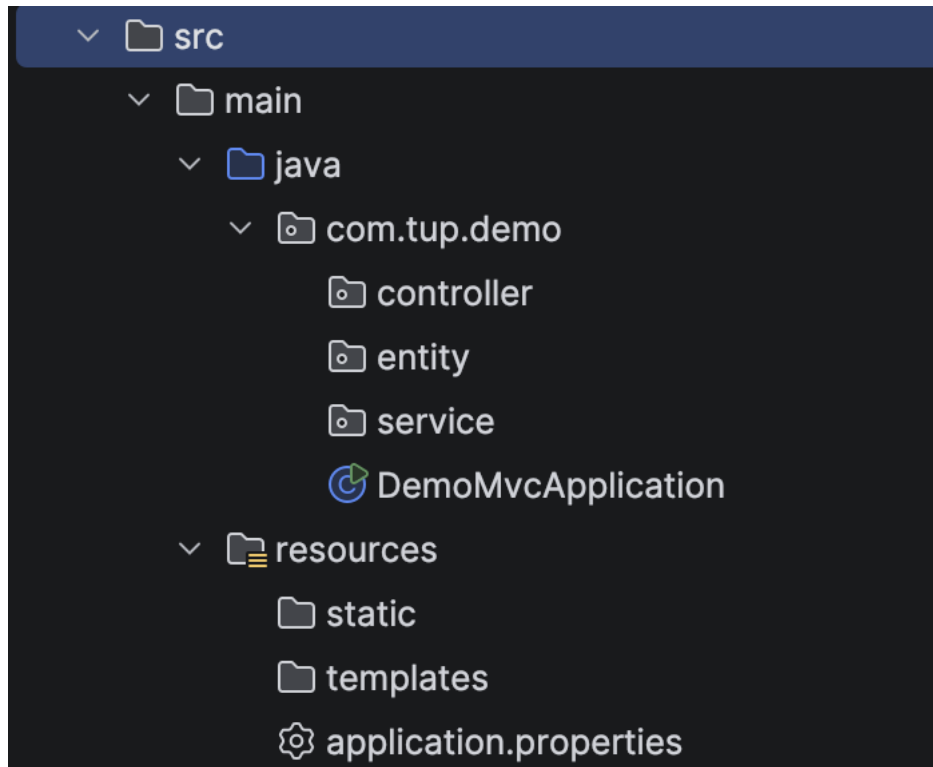
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Thymeleaf

TEMPLATE ENGINES

A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

Struktur Project



Penjelasan:

- **controller** → menangani request
- **entity/model** → representasi data
- **templates** → halaman HTML

Implementasi MVC di Springboot

Komponen	Implementasi di Spring Boot
Model	class data / entity
View	HTML template
Controller	class controller

Request Mapping

- Di Spring WebMVC, untuk melakukan Routing, menggunakan annotation `@RequestMapping` pada method yang ingin kita jadikan sebagai Controller Handler-nya
- <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/bind/annotation/RequestMapping.html>

Membuat Controller

- Buat Kelas HalloController.java pada package controller
- Memberi anotasi @Controller pada kelas yang dibuat
- Buat method halo
- Beri request @RequestMapping di method halo
- Buat File index.html pada folder resources

Hasil Code



```
package com.tup.demo.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class HalloController {

    @RequestMapping(method = RequestMethod.GET, value = "/" )
    public String hallo() {
        return "index";
    }
}
```

RequestMapping “GET”

Digunakan untuk mengambil atau mendapatkan sumber daya dari server. Hanya digunakan untuk membaca data

Penggunaan Request Mapping “GET”

- Modifikasi kelas HalloController.java dengan menambahkan

```
@RequestMapping(method = RequestMethod.GET, value = "/register" )  
public String register() {  
    return "register";  
}
```

- Modifikasi file index.html dengan menambahkan

```
<body>  
    <h1>Halo ini adalah project spring pertama ku</h1>  
  
    <a href="/register">Go to Register Page</a>
```

- Buat file register.html

RequestMapping “POST”

- Digunakan untuk membuat sumber daya dari server

Penggunaan Request Mapping “POST”

- Modifikasi kelas HalloController.java dengan menambahkan

```
@RequestMapping(method = RequestMethod.POST, value = "/register" )
public String processRegister(@RequestParam String name,
                             @RequestParam String email) {
    System.out.println("Name: " + name);
    System.out.println("Email: " + email);
    return "redirect:/";
}
```

Implementasi Model

- Buat Kelas Pojo dengan nama Student.java pada package entity
- Update Controller untuk dapat menyimpan data Student dalam array
- Update Halaman index untuk menampilkan Data Student

```
package com.tup.demo.entity;

public class Student {

    private String name;
    private String email;

    public Student() {}

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Kelas Pojo : Plain Old Java Object

POJO (Plain Old Java Object)

adalah kelas Java biasa yang tidak bergantung pada framework tertentu dan hanya berisi properti (atribut) serta method sederhana seperti getter dan setter.

Hasil Update kelas HalloController.java

```
package com.tup.demo.controller;

import ...

@Controller
public class HalloController {

    private List<Student> studentList=new ArrayList<>();

    @RequestMapping(method = RequestMethod.GET, value = "/" )
    public String hallo(Model model) {
        model.addAttribute("students",studentList);
        return "index";
    }

    @RequestMapping(method = RequestMethod.GET, value = "/register" )
    public String register() {
        return "register";
    }

    @RequestMapping(method = RequestMethod.POST, value = "/register" )
    public String processRegister(@RequestParam String name,
                                @RequestParam String email) {

        Student student=new Student();
        student.setName(name);
        student.setEmail(email);
        studentList.add(student);

        return "redirect:/";
    }
}
```

Hasil Update file index.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="en">
<head>
  <meta charset="UTF-8">
  <title>Halo </title>
</head>
<body>
  <h1>Halo ini adalah project spring pertama ku</h1>

  <a href="/register">Go to Register Page</a>

  <br><br>
  <table border="1">
    <thead>
      <tr>
        <th>Name</th>
        <th>Email</th>
      </tr>
    </thead>

    <tbody>
      <tr th:each="student : ${students}">
        <td th:text="${student.name}"></td>
        <td th:text="${student.email}"></td>
      </tr>
    </tbody>
  </table>
</body>
</html>
```

Implementasi REST API dengan SpringBoot

Rest API

- REST = **Representational State Transfer**, cara komunikasi antar aplikasi lewat HTTP
- Client (frontend/Postman) mengirim request kemudian Server membalas dengan response (biasanya JSON)
- Bersifat stateless : setiap request berdiri sendiri, server tidak menyimpan "ingatan" request sebelumnya

JSON (Java Script Object Notation)

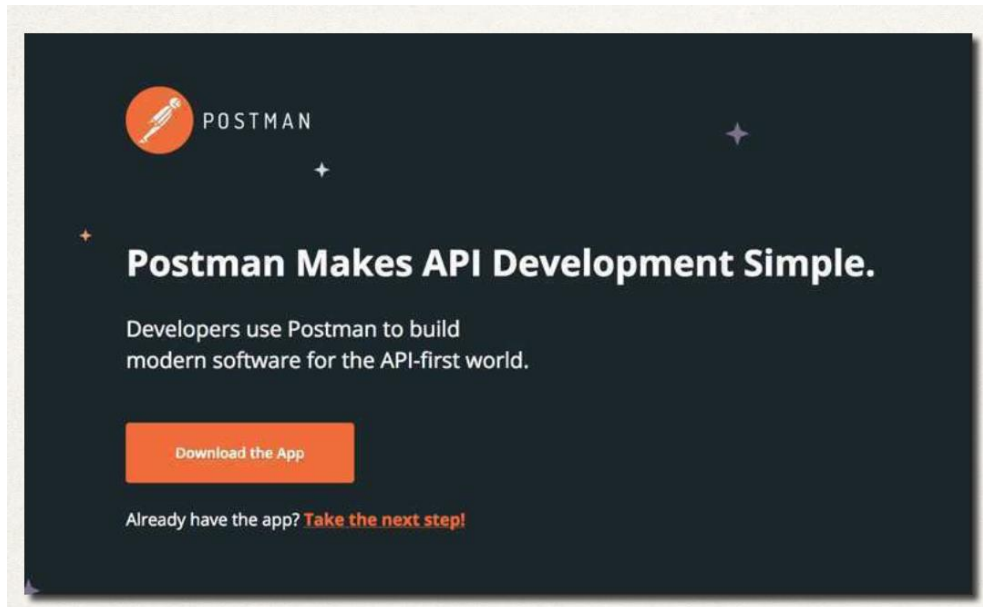
- JSON adalah format teks untuk menyimpan dan mengirim data antar aplikasi. Meskipun namanya "JavaScript", JSON bisa digunakan di semua bahasa pemrograman termasuk Java.
- Contoh :

A code editor window with a dark background and a light gray border. It features three colored window control buttons (red, yellow, green) in the top-left corner. The editor contains a JSON object with the following text:

```
{  
  "id": 1,  
  "nama": "Dany",  
  "prodi": Informatika  
}
```


Install Postman

- Postman adalah aplikasi untuk **mengirim HTTP request ke API** agar kita bisa menguji API tanpa perlu membuat frontend terlebih dahulu.
- <https://www.postman.com/downloads/>



Spring REST





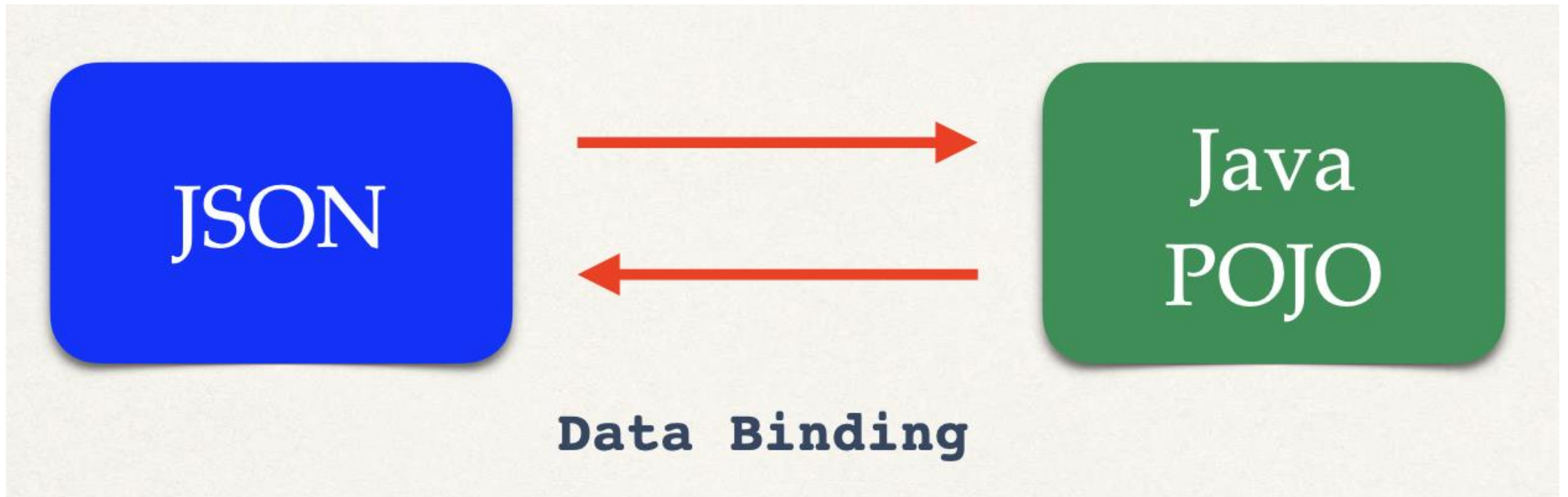
```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello World!";
    }
}
```



Data Binding REST API

Data binding adalah proses konversi otomatis antara JSON dan objek Java (POJO) yang dilakukan Spring Boot.



```
{
  "name" : "Dany Candra",
  "email": "danycandra@telkomuniversity.ac.id"
}
```

```
public class Student {

    private String name;
    private String email;

    public Student() {}

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

}
```

Membuat Rest API

1. Membuat class POJO
2. Membuat Rest Service dengan @RestController
3. Test memanggil REST API

Membuat Class Pojo

```
public class Student {  
  
    private String name;  
    private String email;  
  
    public Student() {}  
  
    public Student(String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getEmail() {  
        return email;  
    }  
  
    public void setEmail(String email) {  
        this.email = email;  
    }  
}
```


Membuat Rest Controller



```
@RestController
@RequestMapping("/api/student")
public class StudentRestController {

    private static List<Student> listStudent = new ArrayList<>(List.of(
        new Student("Dany", "dany@mail.com"),
        new Student("Candra", "candra@mail.com")
    ));

    @GetMapping
    public ResponseEntity<List<Student>> getAllProduk() {
        return ResponseEntity.ok(listStudent);
    }
}
```

Test memanggil REST API

The screenshot shows a REST client interface. At the top, there's a breadcrumb "New Collection > GetAll" and a green button labeled "Panggil REST API". Below this, the request method is set to "GET" and the URL is "localhost:8080/api/student". The response tab is selected, showing a JSON array of two student objects. A green button labeled "Hasil REST API" points to the response data.

HTTP New Collection > GetAll **Panggil REST API**

GET localhost:8080/api/student

Body Cookies Headers 5 Test Results ↺

{ } JSON Preview Visualize

```
1  [
2    {
3      "name": "Dany",
4      "email": "dany@mail.com"
5    },
6    {
7      "name": "Candra",
8      "email": "candra@mail.com"
9    }
10 ]
```

Hasil REST API

@Path Variable

@PathVariable adalah cara mengambil nilai dari segmen URL dan menjadikannya parameter di method Java.

The screenshot shows a REST client interface with a collection named 'GetByName'. A GET request is configured to 'localhost:8080/api/student/Dany', where 'Dany' is highlighted with a green box. A green button labeled 'Panggil REST API' is next to the URL. Below the request, the 'Body' tab is selected, showing a JSON response:

```
{
  "name": "Dany",
  "email": "dany@mail.com"
}
```

 The 'Dany' value in the JSON is highlighted with a green box. A green button labeled 'Hasil REST API' is next to the response. The interface also includes tabs for Cookies, Headers, Test Results, and a 'Visualize' button.

Membuat @PathVariable

1. Tambah Request Mapping di Rest Service dan Tambahkan parameter @PathVariable

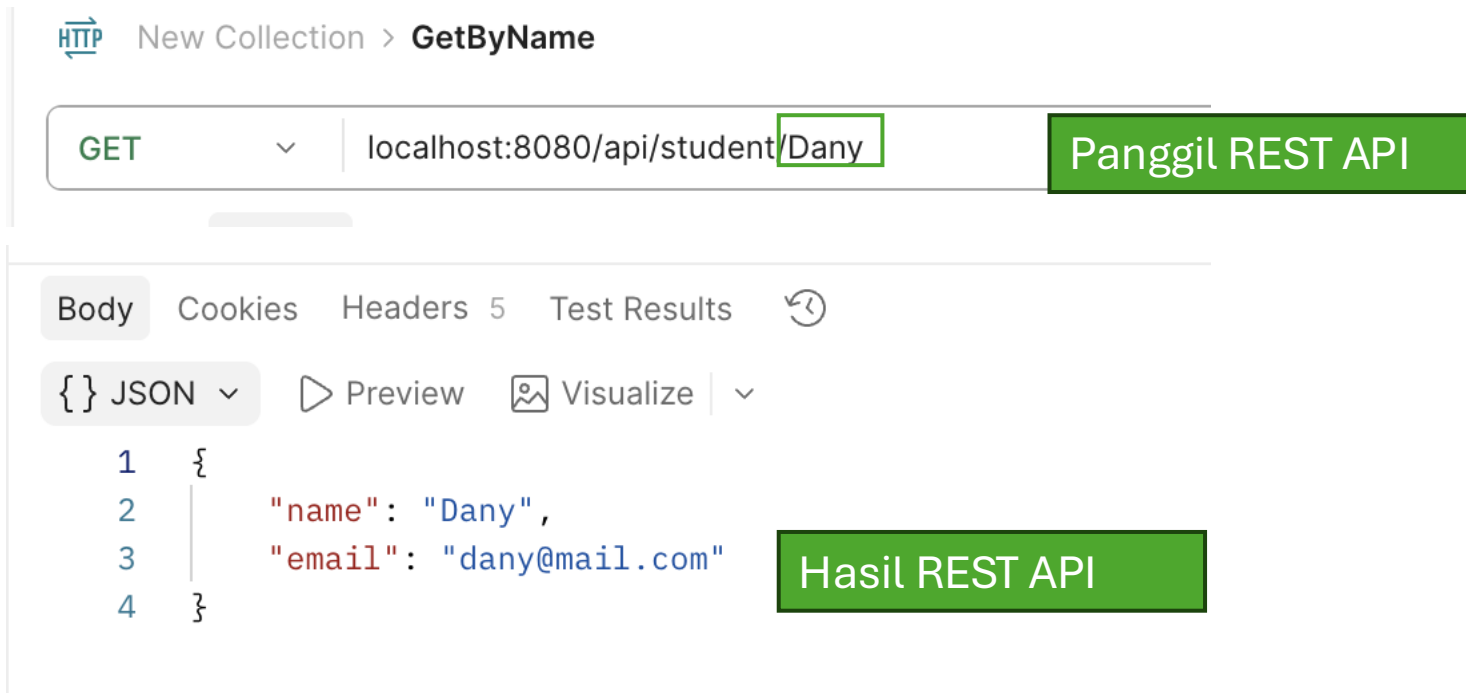
Tambah Request Mapping di Rest Service dan Tambahkan parameter @PathVariable

```
@RestController
@RequestMapping("/api/student")
public class StudentRestController {

    @GetMapping("/{name}")
    public ResponseEntity<Student> getProdukByname(@PathVariable String name) {
        return listStudent.stream()
            .filter(s -> s.getName().equals(name))
            .findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

Test Path Variable

@PathVariable adalah cara mengambil nilai dari segmen URL dan menjadikannya parameter di method Java.



The screenshot shows a REST client interface with a collection named 'GetByName'. A GET request is configured to 'localhost:8080/api/student/Dany', where 'Dany' is highlighted with a green box. A green button labeled 'Panggil REST API' is next to the URL. Below the request, the 'Body' tab is selected, showing a JSON response:

```
{
  "name": "Dany",
  "email": "dany@mail.com"
}
```

 The 'Dany' value in the response is highlighted with a green box. A green button labeled 'Hasil REST API' is next to the response. The interface also includes tabs for Cookies, Headers, Test Results, and a 'Visualize' button.

@RequestBody

@RequestBody adalah anotasi untuk mengambil **data JSON** dari **body request** dan mengkonversinya otomatis menjadi objek Java.

Biasanya Digunakan pada method **POST** dan **PUT**, karena kedua method itu perlu mengirim data ke server.

Contoh Penerapan

HTTP New Collection > Post **Panggil REST API**

POST localhost:8080/api/student

Docs Params Authorization Headers 9 Body ● Script

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw

```
1 {  
2   "name": "Denis",  
3   "email": "denis@mail.com"  
4 }
```

Hasil REST API

Body Cookies Headers 5 Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {  
2   "name": "Denis",  
3   "email": "denis@mail.com"  
4 }
```


Membuat @RequestBody

1. Tambah Request Mapping dengan method post di Rest Service dan Tambahkan parameter @RequestBody

Tambah Request Mapping dengan method post di Rest Service dan Tambahkan parameter @RequestBody

```

@RestController
@RequestMapping("/api/student")
public class StudentRestController {

    @PostMapping
    public ResponseEntity<Student> tambahSiswa(@RequestBody Student student) {
        listStudent.add(student);
        return ResponseEntity.status(HttpStatus.CREATED).body(student);
    }

}
```

Test Request Body

HTTP New Collection > Post **Panggil REST API**

POST localhost:8080/api/student

Docs Params Authorization Headers 9 Body Script

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw

```
1 {  
2   "name": "Denis",  
3   "email": "denis@mail.com"  
4 }
```

Hasil REST API

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {  
2   "name": "Denis",  
3   "email": "denis@mail.com"  
4 }
```